

# Web Engineering 2

Wintersemester 2026/27  
Bachelor Medieninformatik

**Prof. Dr. Jens von Pilgrim**

Stand: 15.9.2026

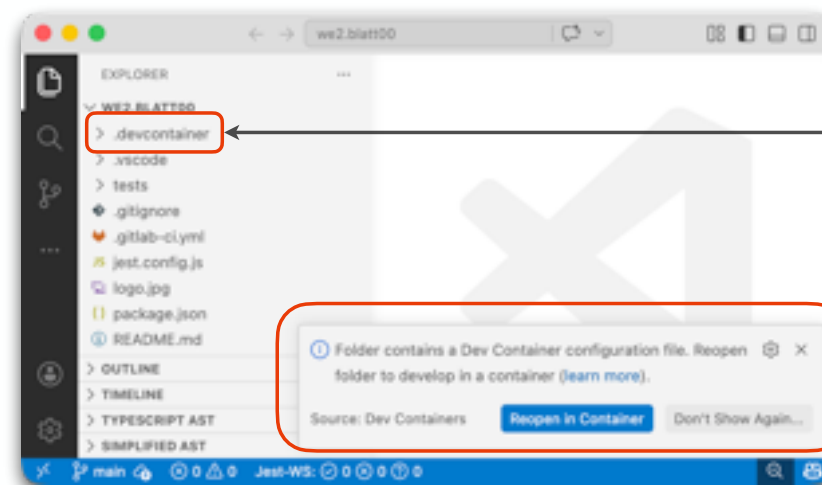


# Supply-Chain Angriffe

# TLDR;

Installieren Sie sich Docker auf Ihrem Rechner und öffnen Sie die Projekte mit VS Code im DevContainer.

Bei den Projekten in WE2 ist das alles schon vorbereitet und Sie werden beim Öffnen von VS Code gefragt, ob Sie das Projekt nicht im DevContainer öffnen wollen.



Dev Container schon konfiguriert

Erscheint automatisch!

# npm install malware

Mittels npm können wir in JavaScript leicht Software von anderen Entwicklern verwenden. In anderen Sprachen gibt es ähnliche Tools und Plattformen (Java - Maven Central, Rust - Cargo, Python - Pip, ...).

Gerade bei JavaScript hat man schnell eine Vielzahl an Paketen Dritter installiert. So findet man bei den Projekten von WE2 ca. 300-500 Pakete im node\_modules-Folder!

**500 Pakete bedeutet für Hacker: 500 Angriffspunkte!**

# Software Supply Chain Attacks

Supply Chain bedeutet allgemein Lieferkette. Software Supply Chains beziehen sich vor allem auf die Bibliotheken und Tools, die beim Bau der eigenen Software verwendet bzw. als Komponenten in die eigene Software einfließen.

Eine Supply Chain Attacke richtet sich nun u.a. auf diese verwendeten Bibliotheken. Wenn der Angreifer es schafft, eine verwendete Bibliothek zu manipulieren, kann er so Schadcode in die produzierte Software einschleusen!

# Kein theoretisches Problem

Quelle hier Socket.dev (über [golem.de](https://golem.de))

- July 22, 2025: npm 'is' Package Hijacked in Expanding Supply Chain Attack (<https://socket.dev/blog/npm-is-package-hijacked-in-expanding-supply-chain-attack>)
  - Packet "is", 2,5 Mio. Downloads / Woche
- August 27, 2025: Nx npm Packages Compromised in Supply Chain Attack Weaponizing AI CLI Tools (<https://socket.dev/blog/nx-packages-compromised>)
  - Packet "nx", 6 Mio. Downloads / Woche
- September 8, 2025: npm Author Qix Compromised via Phishing Email in Major Supply Chain Attack (<https://socket.dev/blog/npm-author-qix-compromised-in-major-supply-chain-attack>)
  - Zugangsdaten von Maintainer "Quix" wurde kompromitiert und damit fast 20 Pakete mit (zusammen) Milliarden von Downlads / Woche
  - Malware versuchte, Kryptozahlungen umzuleiten — zum Glück sehr schlecht programmiert. Aber: **Entwickler waren direkte Angriffsziele!**
- September 15, 2025: Popular Tynicolor npm Package Compromised in Supply Chain Attack Affecting 40+ Packages (<https://socket.dev/blog/>),  
September 16, 2025: Updated and Ongoing Supply Chain Attack Targets CrowdStrike npm Packages (<https://socket.dev/blog/ongoing-supply-chain-attack-targets-crowdstrike-npm-packages>)
  - 40 Pakete verschiedener Maintainer wurden kompromitiert, u.a. @ctrl/tynicolor mit 2 Mio Downloads / Woche
- November 24, 2025: Shai-Hulud 2.0 Supply Chain Attack: 25K+ Repos Exposing Secrets (<https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack>)
  - **700 Pakete kompromitiert**
- **31. März 2026: Bibliothek Axios mit wöchentlich ca. 100 Mio Downloads, wurde kompromittiert.** (<https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>)  
Maintainer wurde mit Social Engineering dazu gebracht, sich eine Malware zu installieren.

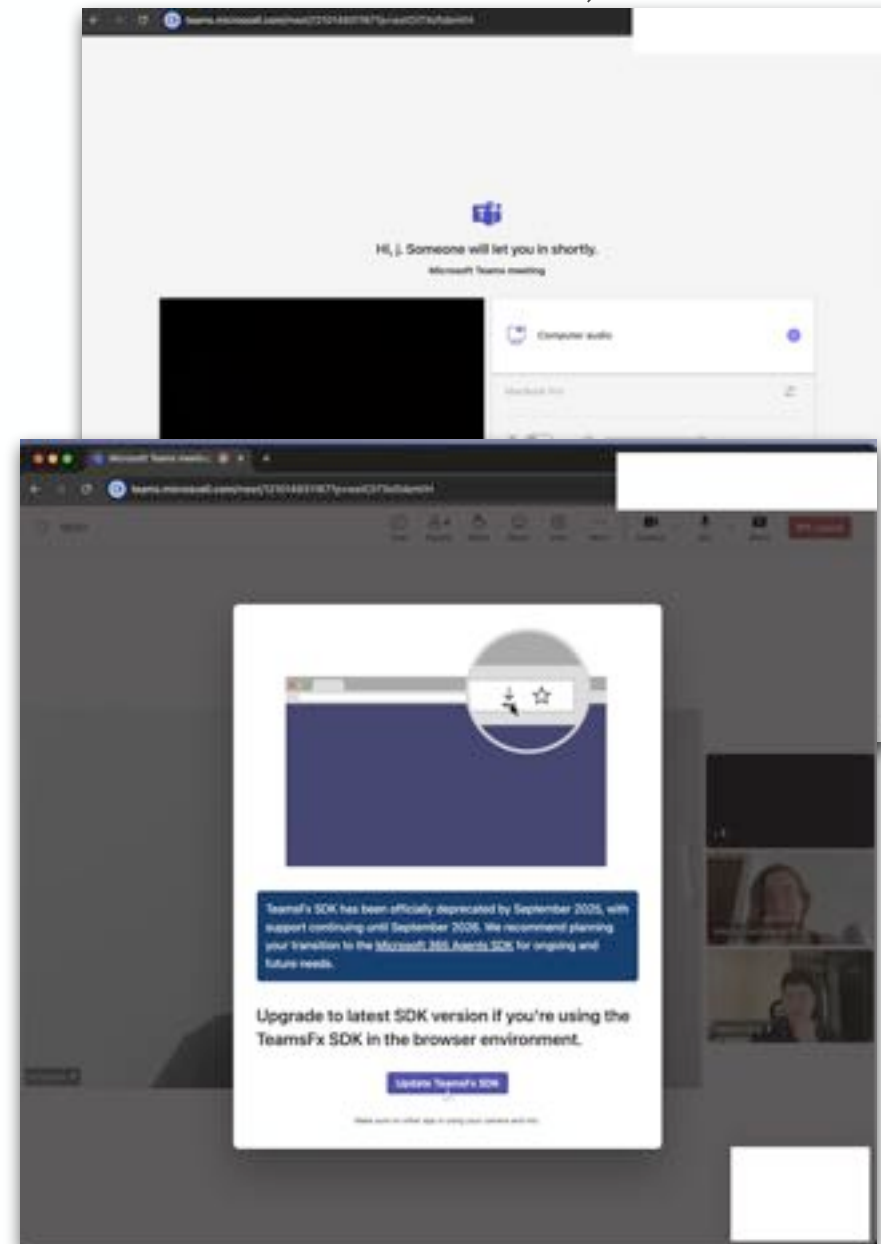
# Social Engineering

Social Engineering bedeutet, dass Personen gezielt und persönlich angesprochen werden und versucht wird, vertrauliche Informationen zu erlangen.

Im Fall des Axios-Hacks wurde ein Maintainer von angeblichen Gründern eines Startups angesprochen. Die Hacker hatten die Seiten eines echten Startups überzeugend nachgebaut und über Wochen einen Kontakt (zunächst über LinkedIn) aufgebaut.

Der eigentliche Angriff kam dann bei einem vermeintlichen Team-Call.

(Quelle: "Post Mortem: axios npm supply chain compromise",  
<https://github.com/axios/axios/issues/10636>;  
Screenshots von tayvano in diesem Thread)



# Package Typosquatting und Package Hallucinations

**Angriff über Typosquatting:** Hacker registrieren Pakete mit Namen, die bekannten Paketen ähneln und quasi Tippfehler dieser darstellen.

[NHW+23] Neupane, Shradha; Holmes, Grant; Wyss, Elizabeth; Davidson, Drew; De Carli, Lorenzo: Beyond typosquatting: an in-depth look at package confusion. In: Proceedings of the 32nd USENIX Conference on Security Symposium. USA : USENIX Association, 2023 (SEC '23). – ISBN 978-1-939133-37-3

| Attack Category | # pkgs | # in attack set |
|-----------------|--------|-----------------|
| Stealing        | 70     | 48              |
| Backdoor        | 9      | 9               |
| Sabotage        | 2      | 1               |
| Cryptojacking   | 2      | 2               |
| Virus           | 1      | 1               |
| Maladvertising  | 2      | 1               |
| PoC             | 1      | 0               |
| Cryptotheft     | 33     | 31              |
| Downloader      | 1      | 0               |
| Confusion       | 2      | 0               |
| Unknown         | 45     | 18              |

**Angriff über Hallucinations:** LLMs erzeugen Paketabhängigkeiten zu Paketen, die es nicht gibt (oder bekannten Paketnamen ähneln). Hacker registrieren Pakete unter solchen Namen.

[SWS+24] Spracklen, Joseph; Wijewickrama, Raveen; Sakib, A. H. M. N.; Maiti, Anindya; Jadliwa-la, Murtuza: We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. DOI 10.48550/ARXIV.2406.10279. In: Computing Research Repository (CoRR), arXiv abs/2406.10279 (2024), Juni

| Programming Language          | No. of Cross-Language Hallucinations |
|-------------------------------|--------------------------------------|
| JavaScript (npm)              | 6,705                                |
| R (CRAN)                      | 293                                  |
| Rust (Cargo)                  | 181                                  |
| Ruby (Rubygems)               | 123                                  |
| PHP (Packagist)               | 47                                   |
| Swift/Objective C (Cocoapods) | 10                                   |
| .NET (Nuget)                  | 9                                    |
| Go (Go)                       | 0                                    |
| Java (Maven)                  | 0                                    |

Prüfen Sie Paketnamen daher immer ganz genau — egal, ob manuell geschrieben oder von KI generiert.

# Schutzmechanismen

Einen 100-prozentigen gegen Angriffe auf die Supply-Chain gibt es nicht — was im Grunde für alle Angriffe gilt.

Mögliche Maßnahmen:

- Verbot von Bibliotheken Dritter oder Notwendigkeit der Zertifizierung — wird bei safety-kritischen Anwendungen auch tatsächlich gemacht!
- Verwendung nicht von npmjs.com (o.ä. Repositories), sondern spezielle Repositories, die von speziellen Security-Anbietern oder In-House auf Malware etc. geprüft werden.

Nachteil: Sehr einschränkend und/oder sehr teuer!

# Schutz des Entwicklerrechners

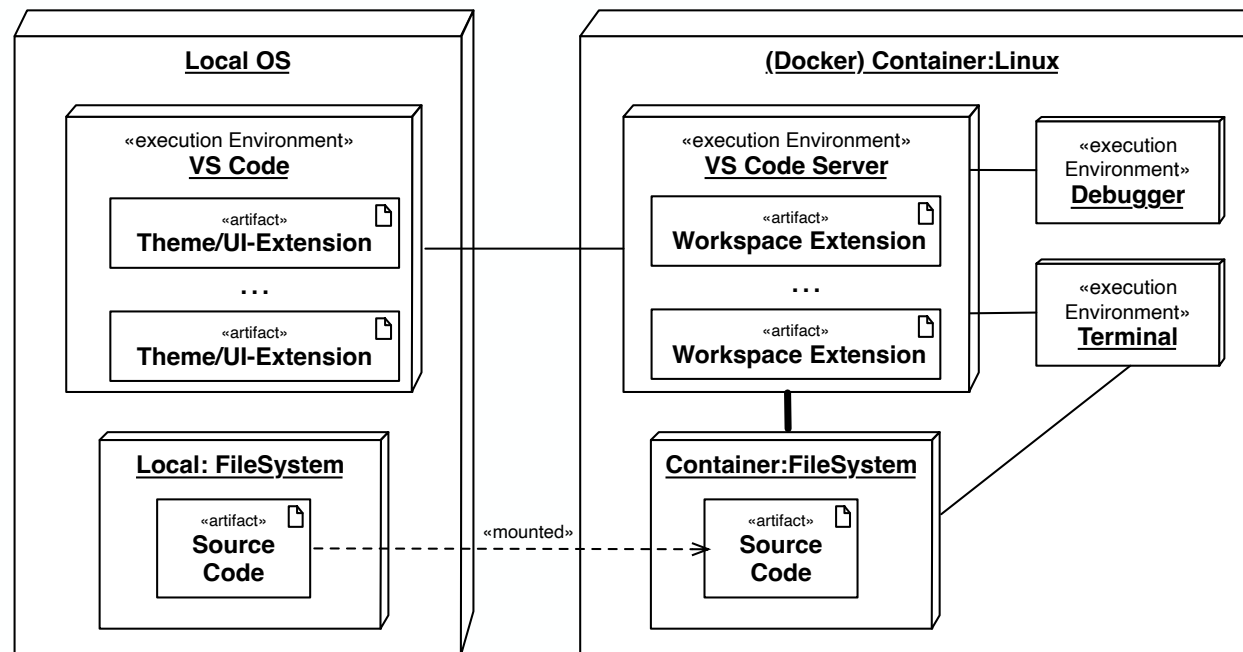
Um sich bei der Auswahl der Pakete nicht einschränken zu müssen und um weiterhin die bekannten Repositories zu verwenden, kann man wenigstens den eigenen Rechner schützen.

- Verwendung eines dedizierten Entwicklerrechners, evtl. speziell zum Ausprobieren neuer Software.
- Verwendung virtueller Entwicklerrechner.

Nachteil: Wer hat schon einen weiteren Rechner rumstehen. Daher:

- Verwendung geschützter Umgebungen, in denen der Workspace bzw. die Software läuft, mittels Docker.

# VS Code mit Dev Container



- Der Workspace liegt auf der lokalen Maschine, wird im Docker-Container gemounted und Skripte auch nur dort ausgeführt.
- VS Code (Server) läuft im Container, VS Code (UI) läuft direkt auf lokaler Maschine.

# VS Code Dev Container, Konfiguration

- Docker muss lokal installiert sein!
- Docker-Image mit der benötigten Entwicklersoftware (außer VS Code), also Node.js, Java etc.
  - Es gibt vorbereitete Images
  - Für das Modul wird ein eigenes Image bereitgestellt. Dieses wird dann auch für die Gitlab-CI verwendet.
- Konfiguration des Dev-Containers: .devcontainer/devcontainer.json
  - Konfiguriert Docker-Image, VS Code Extensions und Settings
  - **In Projektvorlagen schon vorhanden!**
- VS Code entdeckt beim Öffnen eines Workspaces die Dev-Container-Konfiguration, man kann dann VS Code erneut starten, so dass automatisch ein Container gestartet wird, in dem VS Code Server installiert wird und der Workspace eingebunden wird.
  - Alternativ: Installation des CLI devcontainer (über View/Command Palette..., dort “Dev Container: Install devcontainer CLI”) — damit kann dann von Kommandozeile aus direkt ein Ordner im DevContainer geöffnet werden.

siehe auch <https://code.visualstudio.com/docs/devcontainers/containers>

**Docker-Image für Dev  
Container und für  
GitLab-CI ist identisch!**

Aber: Bei Groß-/  
Kleinschreibung muss  
man trotzdem  
aufpassen.

# Sicherheitsrisiken durch genAI

Copilot und andere genAI-Tools ermöglichen es der genAI, Befehle (meist auf der Kommandozeile) auszuführen. Dies wird von genAIs auch häufig gemacht, häufig findet man bspw. Aufrufe von grep oder sed, um Ausgaben zu analysieren. Kritischer wird es dann, wenn die genAI bspw. ein Python-Script schreibt, um irgendetwas zu machen.\*)

Es gibt immer wieder Berichte, wo die genAI etwas macht, womit man nicht gerechnet hat: Löschen von Verzeichnissen, Löschen von Datenbanken etc.

Aus **Sicherheitsgründen** sollte man daher

- Bei der Ausführung von Scripten genau hinschauen. Wenn man nicht weiß, was da passiert, ist es im Grunde grob fahrlässig, wenn man die genAI gewähren lässt.
- Generell sollte man der genAI **nicht erlauben, Git auszuführen**. Das Praktische an Git ist ja, das man im Diff sehr schon sieht, was wirklich verändert wurde. Wenn nun die genAI selbstständig committed, sieht man das nicht mehr (so einfach).
- Um die Gefahr zu mindern, sollte man generell in einem **DevContainer** arbeiten. Dann kann die genAI zumindest (nicht so leicht) auf dem Host Schaden anrichten.

